15/10/26

CO1 -ASSSESMENT TOOL 3

CODING CHALLENGE

NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME:N.Umasri **REG.NO: 192424240**

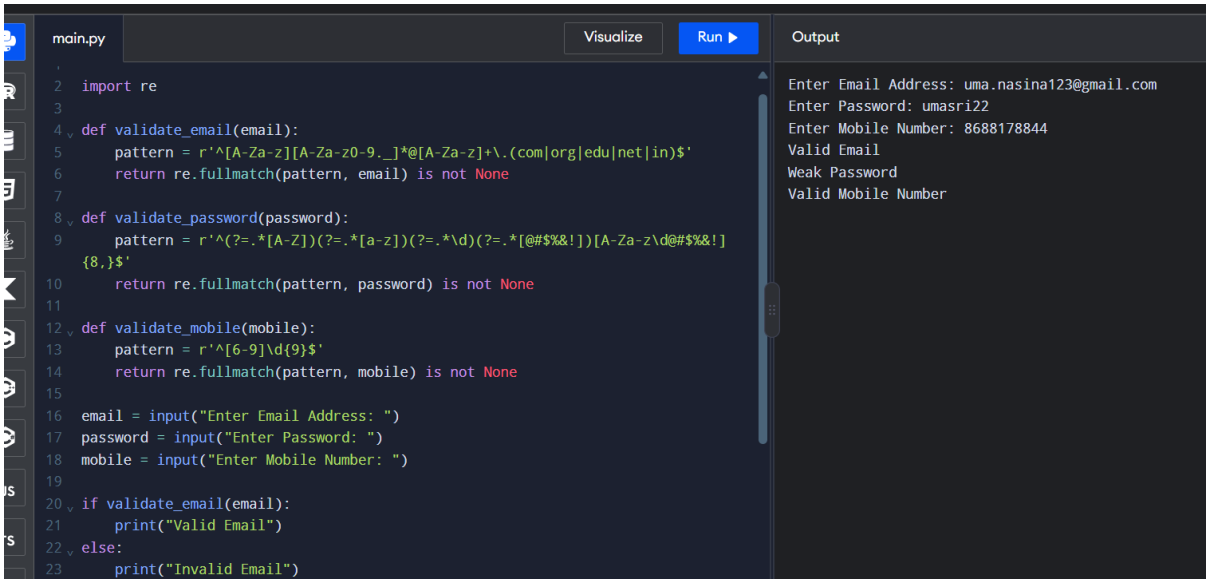
Section 1: Intelligent Email and Password Validator

Code:

```
import re
def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.fullmatch(pattern, email) is not None
def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])[A-Za-z\d@#$%&!]{8,}$'
    return re.fullmatch(pattern, password) is not None
def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return re.fullmatch(pattern, mobile) is not None
email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
if validate_email(email):
    print("Valid Email")
else:
    print("Invalid Email")
```

```
if validate_password(password):
    print("Strong Password")
else:
    print("Weak Password")
if validate_mobile(mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

output:



Section 2: Design a Finite State Automata (FSA) Simulator

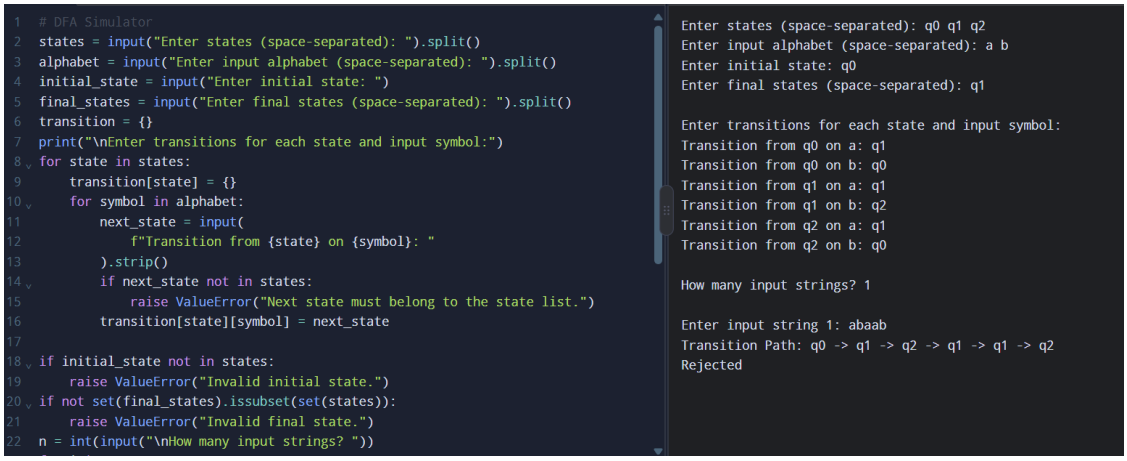
Code:

```
# DFA Simulator
states = input("Enter states (space-separated): ").split()
alphabet = input("Enter input alphabet (space-separated): ").split()
initial_state = input("Enter initial state: ")
final_states = input("Enter final states (space-separated): ").split()
transition = {}
print("\nEnter transitions for each state and input symbol:")
for state in states:
```

```
    transition[state] = {}
    for symbol in alphabet:
        next_state = input(
            f"Transition from {state} on {symbol}: "
        ).strip()
        if next_state not in states:
            raise ValueError("Next state must belong to the state list.")
        transition[state][symbol] = next_state

if initial_state not in states:
    raise ValueError("Invalid initial state.")
if not set(final_states).issubset(set(states)):
    raise ValueError("Invalid final state.")
n = int(input("\nHow many input strings? "))
for i in range(n):
    string = input(f"\nEnter input string {i + 1}: ")
    current_state = initial_state
    path = [current_state]
    valid = True
    for symbol in string:
        if symbol not in alphabet:
            valid = False
            break
        current_state = transition[current_state][symbol]
        path.append(current_state)
    print("Transition Path:", " -> ".join(path))
    if valid and current_state in final_states:
        print("Accepted")
    else:
        print("Rejected")
```

output:



```
# DFA Simulator
states = input("Enter states (space-separated): ").split()
alphabet = input("Enter input alphabet (space-separated): ").split()
initial_state = input("Enter initial state: ")
final_states = input("Enter final states (space-separated): ").split()
transition = {}
print("\nEnter transitions for each state and input symbol:")
for state in states:
    transition[state] = {}
    for symbol in alphabet:
        next_state = input(
            f"Transition from {state} on {symbol}: "
        ).strip()
        if next_state not in states:
            raise ValueError("Next state must belong to the state list.")
        transition[state][symbol] = next_state

if initial_state not in states:
    raise ValueError("Invalid initial state.")
if not set(final_states).issubset(set(states)):
    raise ValueError("Invalid final state.")
n = int(input("\nHow many input strings? "))
```

```
Enter states (space-separated): q0 q1 q2
Enter input alphabet (space-separated): a b
Enter initial state: q0
Enter final states (space-separated): q1

Enter transitions for each state and input symbol:
Transition from q0 on a: q1
Transition from q0 on b: q0
Transition from q1 on a: q1
Transition from q1 on b: q2
Transition from q2 on a: q1
Transition from q2 on b: q0

How many input strings? 1

Enter input string 1: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Rejected
```

Section 3: Smart Pattern Matching Engine using Regular Expressions

Code:

```
import re
text = input("Enter the text to search:\n")
patterns = {
    "1": ("Date", r'\b\d{2}/\d{2}/\d{4}\b'),
    "2": ("Phone Number", r'\b[6-9]\d{9}\b'),
    "3": ("Hashtag", r'(?<!\w)#[A-Za-z0-9_]+'),
    "4": ("Mention", r'(?<!\w)@[A-Za-z0-9_]+'),
}
print("\nUser Menu")
print("1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Word")
choice = input("Enter your choice: ")
if choice in patterns:
```

```
    name, pattern = patterns[choice]
    matches = re.findall(pattern, text)
elif choice == "5":
    prefix = input("Enter prefix: ")
    pattern = r'\b' + re.escape(prefix) + r'\w*'
    matches = re.findall(pattern, text, re.IGNORECASE)
    name = "Prefix"
elif choice == "6":
    suffix = input("Enter suffix: ")
    pattern = r'\b\w*' + re.escape(suffix) + r'\b'
    matches = re.findall(pattern, text, re.IGNORECASE)
    name = "Suffix"
elif choice == "7":
    word = input("Enter word to search: ")
    pattern = r'\b' + re.escape(word) + r'\b'
    matches = re.findall(pattern, text, re.IGNORECASE)
    name = "Word"
else:
    print("Invalid choice")
    matches = []
    name = ""
if choice in ["1", "2", "3", "4", "5", "6", "7"]:
    print("\nSearch Result:", name)
    if matches:
        for match in matches:
            print(match)
    else:
        print("No matching patterns found.")
```

output:

```python
import re
text = input("Enter the text to search:\n")
patterns = {
    "1": ("Date", r'\b\d{2}/\d{2}/\d{4}\b'),
    "2": ("Phone Number", r'\b[6-9]\d{9}\b'),
    "3": ("Hashtag", r'(?<!\w)#[A-Za-z0-9_]+'),
    "4": ("Mention", r'(?<!\w)@[A-Za-z0-9_]+'),
}
print("\nUser Menu")
print("1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Word")
choice = input("Enter your choice: ")
if choice in patterns:
    name, pattern = patterns[choice]
    matches = re.findall(pattern, text)
elif choice == "5":
    prefix = input("Enter prefix: ")
```

```
Enter the text to search:
meeting on 15/10/26

User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
Enter your choice: 2

Search Result: Phone Number
No matching patterns found.
```